

04-ansible 的 playbook

1.1 playbook

playbook，剧本

所谓的 playbook 就是一个文件，里面通过 yaml 语法进行编写，可以在 playbook 中调用多个模块，同时还支持变量的定义和引用、支持判断语句、支持循环语句...

特点：按照顺序来执行剧本中的任务的（这个任务就是要在被控节点执行的任务，一个任务调用一个 ansible 模块），一旦中间有某个任务失败了，那么 ansible 就会停止在这个任务，后面的任务不会去执行了

如果编写了剧本的话，那么执行剧本的时候就不能使用 `ansible` 命令，而需要使用 `ansible-playbook` 命令去执行

不过，`ansible-playbook` 命令和 `ansible` 命令的选项都是通用的

1 playbook 如何编写

案例

YAML | 复制

```
[root@adsl-172-10-0-153 ansible]# cat playbook.yaml
- name: play
  hosts: all
  tasks:
    - name: create lisi user
      user:
        name: lisi
        state: present
    - name: passwd lisi user
      shell: echo 1 | passwd --stdin lisi
```

说明一下：playbook 中的语法格式采用的是 yaml 语法格式

1.1 yaml 语法格式

yaml 语法格式：专门为配置文件编写的一种语法格式，一般都是使用在配置文件上

比如 ansible 中的 playbook，比如 K8S 中的资源文件（pod、deployment...）

1. yaml 语法中大小写严格区分
2. yaml 中，具备层级关系，使用空格来表示层级关系，空格的数量多少无所谓，但是同一个层级下的一定要并齐
3. yaml 中不允许使用 `tab` 键来表示层级关闭，你必须使用空格隔开

1.2 yaml 数据类型

1. 纯量：是 yaml 中最小的单位，是不可以再次分割的单位
2. 列表/数组/序列：一个-就是代表一个列表，列表是一组有次序的内容
在 playbook 中，一个 play 就是一个列表，一个 task 任务就是一个列表
3. 对象/字典：表现形式是键值对 `key: value`
在 playbook 中，调用的模块、模块的参数这些东西都是对象类型

1.3 playbook 的组成和编写

playbook 是由多个 play 组成的，每个 play 可以定义不同的 task 任务，可以定义在不同的主机上执行任务，可以定义变量...

一个 playbook 存在多个 play，我们又称之为 multi play，多个 play

每个 play 的组成是由 4 个部分组成的：

1. target 部分
- 定义这个 play 叫什么名字
- 定义这个 play 中的任务会在哪些主机上执行
- 定义这个 play 中配置参数是什么—>和 ansible.cfg 一模一样，不过优先级比它
2. vars 部分
- 定义变量的，让下面的 task 任务可以引用这些变量
- 可以直接在 play 中定义变量，可以引用外部的变量文件...
3. tasks 部分
- 要在被控节点执行的所有的任务都是在这个 tasks 部分里面写的
- 对于 task 任务来说，一个 task 任务就是调用一个模块
4. handlers 部分
- handlers 在 ansible 中属于触发器，也就是说只有特定的情况下才会执行 handlers
- handlers 其实是属于特殊的 tasks，它下面可以执行多个 task 任务

1.4 剧本案例编写

yaml 文件的后缀尽量是.yaml 或者.yml

第一个：要求剧本名/etc/ansible/play1.yaml，在被控节点实现初始化操作，例如创建一个 demo 用户，uid 是 3450，家目录是/tmp/demo，登录 shell 是/bin/bash，设置用户的密码为 demo，要求这个 demo 用户可以使用 sudo 提权，并且控制节点可以实现免密登录到主机的 demo 用户

YAML

复制

```
[root@adsl-172-10-0-153 ansible]# cat /etc/ansible/play1.yaml
- name: create user
  hosts: all
  tasks:
    - name: create demo user
      user:
        name: demo
        uid: 3450
        home: /tmp/demo
        shell: /bin/bash
        state: present

    - name: modify demo user password
      shell: echo 1 | passwd --stdin demo

    - name: configure demo user sudo
      shell: echo "demo ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers

    - name: mkdir /tmp/demo/.ssh
      file:
        path: /tmp/demo/.ssh
        state: directory
        mode: "0700"
        owner: demo
        group: demo

    - name: configure ssh
      copy:
        src: ~/.ssh/id_rsa.pub
        dest: /tmp/demo/.ssh/authorized_keys
```

YAML

复制

ansible-playbook /etc/ansible/play1.yaml

第二个：要求剧本名/etc/ansible/play2.yaml，在所有的被控节点上执行，要求配置本地 YUM 仓库，然后实现 httpd 软件的安装，并且最终在控制节点上使用 curl 命令访问被控节点主机名的时候，访问 node1 则输出的是 node1 内容，访问 node2 则输出的是 node2 内容

YAML | 复制

```
[root@adsl-172-10-0-153 ansible]# cat play2.yaml
- name: deploy web site
  hosts: all
  tasks:
    - name: clear /etc/yum.repos.d/
      shell: rm -rf /etc/yum.repos.d/*

    - name: mount cdrom
      shell: mount /dev/sr0 /media
      ignore_errors: yes

    - name: configure BaseOS repository
      yum_repository:
        file: dvd
        name: baseos
        description: baseos
        baseurl: file:///media/BaseOS
        gpgcheck: 0
        enabled: 1

    - name: configure AppStream repository
      yum_repository:
        file: dvd
        name: app
        description: app
        baseurl: file:///media/AppStream
        gpgcheck: 0
        enabled: 1

    - name: install httpd pkg
      yum:
        name: httpd
        state: present

    - name: close firewalld
      systemd:
        name: firewalld
        state: stopped
        enabled: no

    - name: close selinux
      shell: setenforce 0

    - name: start httpd service
      systemd:
        name: httpd
        state: restarted
        enabled: yes

- name: node1
  hosts: node1
  tasks:
    - name: modify index.html node1
      copy:
        content: "node1\n"
        dest: /var/www/html/index.html

- name: node2
  hosts: node2
  tasks:
    - name: modify index.html node2
      copy:
```

```
content: "node2\n"
```

如果执行任务报错了，有俩种解决办法

1. 所有的模板都可以采用这种方式，直接忽略这个报错的任务，这样的话，playbook 会继续执行

YAML 复制

```
ignore_errors: yes
```

2. 只针对于命令执行模块有效果，就是添加一个 || true

如果执行任务失败的话，只有这个执行失败的主机才会停止后面任务的执行，其他的主机不受到影响

1.5 handlers 部分补充

handlers 属于特殊的 tasks，是属于 play 四大部分之一，所以他们的层级关系是一样的。

handlers 是触发器，只有特定的情况下才会去执行 handlers 下面的任务

特定的情况：监控一个 task 任务，当这个 **task** 任务的返回状态是 **changed** 的时候，则触发 handlers 任务

无论监听的这个 **task** 任务是什么时候执行的，handlers 下面的任务只会在所有的任务完成最后去执行

简单的 handlers 测试 yaml

YAML 复制

```
- name: play3
  hosts: all
  tasks:
    - name: task1
      shell: ls /etc/passwd
      notify: aaaa

    - name: task2
      shell: ls /etc/group

  handlers:
    - name: aaaa
      shell: echo handlers
```

上面这种写法，即使监听了某个 task 任务，即使在 handlers 下面编写了很多很多的任务，但是最后执行 handlers 任务的时候，只会执行最后一个 handlers 任务

如果要想执行 handlers 下面多个 task 任务，你需要加入一个关键字 listen

YAML 复制

```
- name: play3
  hosts: all
  tasks:
    - name: task1
      shell: ls /etc/passwd
      notify: aaaa

    - name: task2
      shell: ls /etc/group

  handlers:
    - name: handlers1
      listen: aaaa
      shell: echo handlers

    - name: handlers2
      listen: aaaa
      shell: echo aaaa
```

如果执行任务中，有任何一个任务报错，即使监听的任务是 changed 状态，handlers 任务也不会触发

```
[root@adsl-172-10-0-153 ansible]# cat play3.yaml
```

```
- name: play3
  hosts: all
  tasks:
    - name: task1
      shell: ls /etc/passwd
      notify: aaaa

    - name: task2
      shell: lss /etc/group

  handlers:
    - name: aaaa
      shell: echo handlers
```

我们可以看到，明明监听的任务是 changed，但是 handlers 没有触发

如果需要强制执行 handlers 任务，加上 force_handlers

```
- name: play3
  hosts: all
  force_handlers: yes
  tasks:
    - name: task1
      shell: ls /etc/passwd
      notify: aaaa

    - name: task2
      shell: lss /etc/group

  handlers:
    - name: aaaa
      shell: echo handlers
```

强制执行的前提，是你监听的那个任务状态是 changed

什么时候使用 handlers 的场景

在部署 web 服务的场景下，我们可以使用 notify 监听安装某个软件包的任务，然后最后把启动这个服务的任务写到 handlers 中

